

prosperity prognosticator: machine learning for startup success prediction

TEAM ID:LTVIP2026TMIDS34580

TEAM MEMBERS:

PEDHAPUDI SHAKTI VAMSI-22MH1A0547

POLIMERU SAI KIRAN-22MH1A0552

PINISETTI BALU-22MH1A0551

POLAMATI SURES-22MH1A0553

1. Introduction

In today's rapidly evolving entrepreneurial ecosystem, startups play a critical role in economic growth and innovation. However, a significant percentage of startups fail due to poor funding strategies, weak market positioning, or insufficient planning.

This project aims to develop a Machine Learning-based prediction system that determines whether a startup is likely to succeed or fail based on historical startup data. The system uses a Random Forest Classifier model trained on real startup attributes and provides predictions through a web-based interface built using Flask.

2. Problem Statement

Startup failure is a major risk for investors and entrepreneurs. Predicting startup success manually is difficult due to multiple influencing factors such as:

- Funding rounds
- Total funding amount
- Number of milestones

- Number of investors
- Startup age
- Relationships
- Category
- Location

The goal of this project is:

To build a predictive system that classifies startups as successful or unsuccessful using machine learning techniques.

3. Objectives

The primary objectives of this project are:

1. To analyze startup dataset.
 2. To clean and preprocess data.
 3. To perform exploratory data analysis (EDA).
 4. To build a classification model using Random Forest.
 5. To evaluate model performance.
 6. To deploy the model using Flask.
 7. To create a web interface for prediction.
-

4. System Architecture

Architecture Flow:

Dataset → Data Cleaning → Feature Engineering → Model Training →
Model Evaluation → Model Saving → Flask App → Web Interface → Prediction

Components:

1. Dataset (CSV file)
2. Python (Pandas, NumPy)
3. Scikit-learn

4. Random Forest Algorithm
5. Flask Web Framework
6. HTML (Frontend)
7. Joblib (Model Saving)

5. Dataset Description

The dataset contains startup-related features such as:

Feature	Description
founded_year	Year startup was founded
age_first_funding_year	Age at first funding
age_last_funding_year	Age at last funding
funding_total_usd	Total funding received
relationships	Number of relationships
milestones	Number of milestones
has_VC	Venture capital funding
has_angel	Angel funding
status	Target variable (Success/Failure)

Target Variable:

- 1 → Successful
- 0 → Failed

6. Data Preprocessing

6.1 Handling Missing Values

- Checked null values using:

```
data.isnull().sum()
```

- Dropped or imputed missing values where necessary.

6.2 Handling Categorical Variables

Columns like:

- state_code
- category

Were encoded using:

```
pd.get_dummies()
```

6.3 Feature Selection

Removed irrelevant columns such as:

- category_code
 - is_software
 - is_mobile
 - is_enterprise
 - etc.
-

7. Exploratory Data Analysis (EDA)

7.1 State vs Status

Analyzed startup success distribution by state.

7.2 Category Evolution

Used crosstab and line plots to analyze growth of startup categories over years.

7.3 Funding Distribution

Boxplots were created to analyze funding patterns across years.

7.4 Correlation Heatmap

Used:

```
sns.heatmap(corr, cmap='coolwarm', annot=True)
```

To identify relationships between numerical features.

8. Model Selection

We selected:

✓ Random Forest Classifier

Why Random Forest?

- Handles non-linear relationships
- Reduces overfitting
- Works well with large datasets
- High accuracy
- Robust to noise

9. Model Training

9.1 Splitting Dataset

```
X = data_encoded.drop('status', axis=1)
y = data_encoded['status']

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)
```

9.2 Training Model

```
model = RandomForestClassifier()
model.fit(x_train, y_train)
```

10. Hyperparameter Tuning

Used GridSearchCV:

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 4, 6],
    'min_samples_leaf': [1, 2, 3],
}
```

```
'bootstrap': [True, False]
}
```

This improves model performance by selecting optimal parameters.

11. Model Evaluation

Metrics Used:

- Accuracy Score
- Confusion Matrix
- Classification Report

```
accuracy_score(y_test, y_pred_test)
classification_report(y_test, y_pred_test)
```

Observations:

- Training Accuracy: ~ High
 - Testing Accuracy: Acceptable
 - Slight overfitting observed
-

12. Model Saving

Used Joblib to save model:

```
joblib.dump(model, 'random_forest_model.pkl')
```

This allows reuse without retraining.

13. Web Application Development

Framework Used:

Flask

app.py Structure:

1. Load trained model
2. Create route for homepage

3. Accept user input
 4. Convert input to array
 5. Predict result
 6. Display output
-

14. Frontend Design

Created using:

- HTML
- Bootstrap (for styling)

Input Fields:

- Age at First Funding
- Age at Last Funding
- Number of Relationships
- Number of Funding Rounds
- Total Funding
- Milestones
- Average Participants

Prediction Button:

Calls Flask backend and returns result.

15. Results

The system successfully:

- Accepts startup parameters
- Predicts success probability
- Displays result instantly

Example:

Input:

- Funding: \$5,000,000
- Milestones: 4
- Relationships: 10

Output:

Startup is likely to succeed.

16. Advantages

- Fast predictions
- Easy to use
- Good accuracy
- Useful for investors
- Reduces risk analysis time

17. Limitations

- Model depends on historical data
- Limited feature scope
- Overfitting possible
- Real-world unpredictability

18. Future Enhancements

- Deploy on cloud (Heroku / AWS)
- Add probability score
- Improve UI design
- Add data visualization dashboard
- Use advanced models (XGBoost)

- Add real-time investor analysis
-

19. Technologies Used

Technology Purpose

Python	Programming
Pandas	Data analysis
NumPy	Numerical operations
Scikit-learn	ML algorithms
Seaborn	Visualization
Matplotlib	Visualization
Flask	Web backend
HTML/CSS	Frontend
Joblib	Model saving

20. Conclusion

This project demonstrates how machine learning can assist in predicting startup success using historical data. The Random Forest classifier performed well in classification tasks, and deployment using Flask created a practical real-world application.

The system can help investors, venture capitalists, and entrepreneurs make data-driven decisions.